

**LAPORAN PRATIKUM STRUKTUR DATA
BINARY TREE DAN GRAPH DENGAN BAHASA JAVA**



Dosen Pengampu:
Wahyudi Dr.. S.T. M.T.

Disusun Oleh:
Kevin Maulana Sempri
2511531019

**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2025**

Kata Pengantar

Puji syukur penulis ucapkan kepada Allah SWT karena penulis dapat menyelesaikan laporan praktikum ini dengan judul "Binary Tree dan Graph Dengan Bahasa Java" dengan tepat waktu. Laporan ini dibuat sebagai salah satu untuk memenuhi nilai praktikum pada mata Struktur Data. Laporan ini berfokus pada studi kasus Pohon Biner dan Graf dengan menggunakan Bahasa Java. Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna, oleh karena itu penulis mengucapkan banyak terima kasih kepada Bapak / Ibu dosen dan para asisten, serta rekan-rekan mahasiswa yang telah banyak membantu penulis, dalam memberikan arahan & ilmu. Untuk perbaikan & kemajuan di masa yang akan datang, penulis mengharapkan kritik & saran yang membangun. Semoga laporan ini bisa memberikan manfaat & wawasan bagi pembaca, khususnya pada studi kasus Pohon Biner dan Graf menggunakan Bahasa Java. Akhir kata, penulis ucapkan terima kasih kepada semuanya.

DAFTAR PUSTAKA

KATA PENGANTAR	i
DAFTAR ISI	ii
DAFTAR PUSTAKA	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Pratikum.....	1
1.3 Manfaat Pratikum.....	1
BAB II HASIL DAN PEMBAHASAN.....	2
2.1 Waktu Tugas.....	2
2.2 Cara Kerja.....	2
2.3 Hasil.....	3
2.4 Pembahasan	7
DAFTAR PUSTAKA.....	

Bab 1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut adanya sistem penyimpanan data yang terstruktur dan mudah diakses. Struktur Data menjadi salah satu komponen penting dalam pengelolaan informasi, baik di bidang pendidikan, bisnis, maupun pemerintahan. Eclipse IDE merupakan salah satu perangkat lunak yang menyediakan fasilitas untuk merancang, mengelola, dan memanipulasi kode Bahasa pemrograman Java yang relatif mudah dipahami. Melalui praktikum ini, diharapkan mampu memahami Struktur Data Pohon Biner dan Struktur Data Graf serta mengimplementasikan studi kasus dengan menggunakan Bahasa Java.

1.2 Tujuan praktikum

1. Memahami konsep dasar struktur data dengan tipe data abstrak Binary Tree dan Graph.
2. Mempelajari cara membuat struktur data Binary Tree dan Graph dengan menggunakan Bahasa Java.
3. Mengimplementasikan struktur data sesuai kebutuhan studi kasus.
4. Melatih keterampilan dalam mengelola data secara sistematis dan efisien.

1.3 Manfaat praktikum

1. Memberikan pengalaman langsung dalam merancang dan mengelola struktur data.
2. Menambah wawasan tentang penggunaan struktur data dengan menggunakan Bahasa Java.
3. Mempersiapkan mahasiswa dengan keterampilan praktis yang dapat diterapkan dalam dunia kerja.
4. Membantu memahami pentingnya integritas dan konsistensi data dalam sistem informasi.

Bab 2 Hasil dan Pembahasan

Pada bab ini saya akan menampilkan hasil praktikum yang telah dilaksanakan beserta pembahasannya

2.1 Waktu dan Tempat

Praktikum ini dilaksanakan pada hari Senin tanggal 5 Mei 2026 Dilaksanakan di laboratorium Jurusan Informatika, Fakultas Teknologi Infomarsi, Universitas Andalas.

2.2 Cara Kerja

Praktikan akan membuat sebuah program yang berupa Bahasa Pemrograman Java. Perintahnya adalah membuat kode untuk Abstract Data Types (Tipe data abstrak) dan kode untuk Driver (memanggil ADT tersebut)

2.3 Hasil

Kode Java Node:

```
package pekan9_2511531019;

public class Node_2511531019 {
    String name_1019;
    int x_1019, y_1019;
    boolean visited_1019;

    public Node_2511531019(String name_1019, int x_1019, int y_1019)
    {
        this.name_1019 = name_1019;
        this.x_1019 = x_1019;
        this.y_1019 = y_1019;
        this.visited_1019 = false;
    }

    @Override
    public String toString() {
        return name_1019;
    }
}
```

Kode Java Graph:

```
package pekan9_2511531019;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
```

```

public class Graph_2511531019 {
    private Map<Node_2511531019, List<Node_2511531019>> adjacencyList_1019;

    public Graph_2511531019() {
        adjacencyList_1019 = new HashMap<>();
    }

    public void addNode_2511531019(Node_2511531019 node_1019) {
        adjacencyList_1019.putIfAbsent(node_1019, new ArrayList<>());
    }

    public void addEdge_2511531019(Node_2511531019 source_1019, Node_2511531019
destination_1019) {
        adjacencyList_1019.get(source_1019).add(destination_1019);
        adjacencyList_1019.get(destination_1019).add(source_1019);
    }

    public Map<Node_2511531019, List<Node_2511531019>> getAdjacencyList_2511531019() {
        return adjacencyList_1019;
    }

    public void resetGraph_2511531019() {
        for (Node_2511531019 node_1019 : adjacencyList_1019.keySet()) {
            node_1019.visited_1019 = false;
        }
    }
}

```

Kode Java AlgoritmaPencarian:

```

package pekan9_2511531019;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Stack;

public class AlgoritmaPencarian_2511531019 {
    public static HasilPencarian_2511531019 BFS_2511531019(
        Graph_2511531019 graph_1019, Node_2511531019 start_1019, Node_2511531019
goal_1019) {
        Queue<Node_2511531019> queue_1019 = new LinkedList<>();
        Map<Node_2511531019, Node_2511531019> parent_1019 = new HashMap<>();
        List<Node_2511531019> visitedOrder_1019 = new ArrayList<>();
        queue_1019.add(start_1019);
        start_1019.visited_1019 = true;
        while (!queue_1019.isEmpty()) {
            Node_2511531019 current_1019 = queue_1019.poll();
            visitedOrder_1019.add(current_1019);
            if (current_1019 == goal_1019) {
                break;
            }
        }
    }
}

```

```

        for (Node_2511531019 neighbor_1019 :
graph_1019.getAdjacencyList_2511531019().get(current_1019)) {
            if (!neighbor_1019.visited_1019) {
                neighbor_1019.visited_1019 = true;
                parent_1019.put(neighbor_1019, current_1019);
                queue_1019.add(neighbor_1019);
            }
        }
    }
    List<Node_2511531019> path_1019 = buildPath_2511531019(parent_1019, start_1019,
goal_1019);
    return new HasilPencarian_2511531019(path_1019, visitedOrder_1019);
}

public static HasilPencarian_2511531019 DFS_2511531019(
    Graph_2511531019 graph_1019, Node_2511531019 start_1019, Node_2511531019
goal_1019) {
    Stack<Node_2511531019> stack_1019 = new Stack<>();
    Map<Node_2511531019, Node_2511531019> parent_1019 = new HashMap<>();
    List<Node_2511531019> visitedOrder_1019 = new ArrayList<>();
    stack_1019.push(start_1019);
    start_1019.visited_1019 = true;

    while (!stack_1019.isEmpty()) {
        Node_2511531019 current_1019 = stack_1019.pop();
        visitedOrder_1019.add(current_1019);
        if (current_1019 == goal_1019) {
            break;
        }
    }

    for (Node_2511531019 neighbor_1019 :
graph_1019.getAdjacencyList_2511531019().get(current_1019)) {
        if (!neighbor_1019.visited_1019) {
            neighbor_1019.visited_1019 = true;
            parent_1019.put(neighbor_1019, current_1019);
            stack_1019.push(neighbor_1019);
        }
    }
}
    List<Node_2511531019> path_1019 = buildPath_2511531019(parent_1019, start_1019,
goal_1019);
    return new HasilPencarian_2511531019(path_1019, visitedOrder_1019);
}

private static List<Node_2511531019> buildPath_2511531019(
    Map<Node_2511531019, Node_2511531019> parent_1019,
    Node_2511531019 start_1019, Node_2511531019 goal_1019) {
    List<Node_2511531019> path_1019 = new ArrayList<>();
    Node_2511531019 current_1019 = goal_1019;
    while (current_1019 != null) {
        path_1019.add(current_1019);
        current_1019 = parent_1019.get(current_1019);
    }
    Collections.reverse(path_1019);
    if (path_1019.get(0) != start_1019) {
        return new ArrayList<>();
    }
}

```

```

        return path_1019;
    }
}

```

Kode Java HasilPencarian:

```

package pekan9_2511531019;

import java.util.List;

public class HasilPencarian_2511531019 {
    List<Node_2511531019> path_1019;
    List<Node_2511531019> visitedOrder_1019;

    public HasilPencarian_2511531019(
        List<Node_2511531019> path_1019, List<Node_2511531019> visitedOrder_1019) {
        this.path_1019 = path_1019;
        this.visitedOrder_1019 = visitedOrder_1019;
    }
}

```

Kode Java GUI Panel GraphPanel:

```

package pekan9_2511531019;

import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.util.ArrayList;
import java.util.List;

import javax.swing.JPanel;

public class GraphPanel_2511531019 extends JPanel {

    private static final long serialVersionUID = 1L;

    private Graph_2511531019 graph_1019;
    private List<Node_2511531019> path_1019 = new ArrayList<>();

    /**
     * Create the panel.
     */
    public GraphPanel_2511531019(Graph_2511531019 graph_1019) {
        this.graph_1019 = graph_1019;
        setBackground(Color.WHITE);
    }

    public void displayPath_2511531019(List<Node_2511531019> path_1019) {
        this.path_1019 = path_1019;
        repaint();
    }

    @Override
    protected void paintComponent(Graphics g_1019) {
        super.paintComponent(g_1019);
    }
}

```



```

Graphics2D g2_1019 = (Graphics2D) g_1019;

// gambar edge
for (Node_2511531019 node_1019 : graph_1019.getAdjacencyList_2511531019().keySet())
{
    for (Node_2511531019 neighbor_1019 :
        graph_1019.getAdjacencyList_2511531019().get(node_1019)) {

        g2_1019.drawLine(node_1019.x_1019,
            node_1019.y_1019, neighbor_1019.x_1019, neighbor_1019.y_1019);

    }
}

// gambar node
for (Node_2511531019 node_1019 : graph_1019.getAdjacencyList_2511531019().keySet())
{
    if (node_1019.visited_1019) {
        g2_1019.setColor(Color.ORANGE);
    } else {
        g2_1019.setColor(Color.CYAN);
    }
    g2_1019.fillOval(node_1019.x_1019 - 15, node_1019.y_1019 - 15, 30, 30);
    g2_1019.setColor(Color.BLACK);
    g2_1019.drawOval(node_1019.x_1019 - 15, node_1019.y_1019 - 15, 30, 30);
    g2_1019.drawString(node_1019.name_1019,
        node_1019.x_1019 - 20, node_1019.y_1019 - 20);
}

// gambar jalur
g2_1019.setStroke(new BasicStroke(4));
g2_1019.setColor(Color.RED);
for (int i_1019 = 0; i_1019 < path_1019.size() - 1; i_1019++) {
    Node_2511531019 a_1019 = path_1019.get(i_1019);
    Node_2511531019 b_1019 = path_1019.get(i_1019 + 1);
    g2_1019.drawLine(a_1019.x_1019, a_1019.y_1019, b_1019.x_1019, b_1019.y_1019);
}
}
}

```

Kode Java GUI PetaTempatWisata:

```

package pekan9_2511531019;

import java.awt.BorderLayout;
//import java.awt.EventQueue;

import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
//import javax.swing.border.EmptyBorder;

public class PetaTempatWisata_2511531019 extends JFrame {

```

```

        private static final long serialVersionUID = 1L;
        // private JPanel contentPane;
        private Graph_2511531019 graph_1019;
private GraphPanel_2511531019 graphPanel_1019;

private JComboBox<Node_2511531019> startBox_1019;
private JComboBox<Node_2511531019> goalBox_1019;
private JTextArea resultArea_1019;

/**
 * Launch the application.
 */
/**
public static void main(String[] args) {
    EventQueue.invokeLater(new Runnable() {
        public void run() {
            try {
                PetaTempatWisata_2511531019 frame = new
PetaTempatWisata_2511531019();
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
}
*/

/**
 * Create the frame.
 */
public PetaTempatWisata_2511531019() {
    /*
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane = new JPanel();
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane);
    */
    graph_1019 = createGraph_2511531019();

    setTitle("BFS dan DFS Peta Wisata");
    setSize(1000,700);
    setDefaultCloseOperation(EXIT_ON_CLOSE);

    graphPanel_1019 = new GraphPanel_2511531019(graph_1019);

    startBox_1019 = new JComboBox<>();
    goalBox_1019 = new JComboBox<>();

    for (Node_2511531019 node_1019 : graph_1019.getAdjacencyList_2511531019().keySet()) {
        startBox_1019.addItem(node_1019);
        goalBox_1019.addItem(node_1019);
    }

    JButton bfsBtn_1019 = new JButton("BFS");

```

```

        JButton dfsBtn_1019 = new JButton("DFS");
        JButton resetBtn_1019 = new JButton("Reset");

        resultArea_1019 = new JTextArea(10,30);

        JPanel topPanel_1019 = new JPanel();

        topPanel_1019.add(new JLabel("Mulai"));
        topPanel_1019.add(startBox_1019);

        topPanel_1019.add(new JLabel("Tujuan"));
        topPanel_1019.add(goalBox_1019);

        topPanel_1019.add(bfsBtn_1019);
        topPanel_1019.add(dfsBtn_1019);
        topPanel_1019.add(resetBtn_1019);

        add(topPanel_1019, BorderLayout.NORTH);
        add(graphPanel_1019, BorderLayout.CENTER);
        add(new JScrollPane(resultArea_1019), BorderLayout.SOUTH);

        bfsBtn_1019.addActionListener(e_1019 -> runBFS_2511531019());

        dfsBtn_1019.addActionListener(e_1019 -> runDFS_2511531019());

        resetBtn_1019.addActionListener(e_1019 -> resetGraph_2511531019());

        setVisible(true);
    }

    private void runBFS_2511531019() {

        graph_1019.resetGraph_2511531019();

        Node_2511531019 start_1019 = (Node_2511531019) startBox_1019.getSelectedItemAt();
        Node_2511531019 goal_1019 = (Node_2511531019) goalBox_1019.getSelectedItemAt();

        HasilPencarian_2511531019 result_1019 =
            AlgoritmaPencarian_2511531019.BFS_2511531019(graph_1019, start_1019,
goal_1019);

        graphPanel_1019.displayPath_2511531019(result_1019.path_1019);

        showResult_2511531019("BFS", result_1019);
    }

    private void runDFS_2511531019() {

        graph_1019.resetGraph_2511531019();

        Node_2511531019 start_1019 = (Node_2511531019) startBox_1019.getSelectedItemAt();
        Node_2511531019 goal_1019 = (Node_2511531019) goalBox_1019.getSelectedItemAt();

        HasilPencarian_2511531019 result_1019 =
            AlgoritmaPencarian_2511531019.DFS_2511531019(graph_1019, start_1019,
goal_1019);

```

```

graphPanel_1019.displayPath_2511531019(result_1019.path_1019);

showResult_2511531019("DFS", result_1019);
}

private void showResult_2511531019(String method_1019, HasilPencarian_2511531019
result_1019) {
    StringBuilder sb_1019 = new StringBuilder();
    sb_1019.append("Metode : ")
        .append(method_1019)
        .append("\n\n");

    sb_1019.append("Urutan Node Dikunjungi:\n");

    for (Node_2511531019 n_1019 : result_1019.visitedOrder_1019) {
        sb_1019.append(n_1019.name_1019).append(" -> ");
    }

    sb_1019.append("\n\n");

    sb_1019.append("Jalur Ditemukan:\n");

    for (Node_2511531019 n_1019 : result_1019.path_1019) {
        sb_1019.append(n_1019.name_1019).append(" -> ");
    }

    sb_1019.append("\n\n");
    sb_1019.append("Jumlah Node Dieksplorasi : ")
        .append(result_1019.visitedOrder_1019.size());

    resultArea_1019.setText(sb_1019.toString());
}

private void resetGraph_2511531019() {
    graph_1019.resetGraph_2511531019();
    graphPanel_1019.displayPath_2511531019(new java.util.ArrayList<>());
    resultArea_1019.setText("");
}

private Graph_2511531019 createGraph_2511531019() {
    Graph_2511531019 g_1019 = new Graph_2511531019();

    Node_2511531019 airManis_1019 =
        new Node_2511531019("Pantai Air Manis",100,300);
    Node_2511531019 jamGadang_1019 =
        new Node_2511531019("Jam Gadang",300,100);
    Node_2511531019 harau_1019 =
        new Node_2511531019("Lembah Harau",450,100);
    Node_2511531019 maninjau_1019 =
        new Node_2511531019("Danau Maninjau",350,250);
    Node_2511531019 pagaruyung_1019 =
        new Node_2511531019("Istana Pagaruyung",500,300);
    Node_2511531019 pantaiPadang_1019 =
        new Node_2511531019("Pantai Padang",150,150);
    Node_2511531019 bukittinggi_1019 =
        new Node_2511531019("Bukit Tinggi",300,250);
    Node_2511531019 sianok_1019 =

```

```

        new Node_2511531019("Ngarai Sianok",400,350);
Node_2511531019 pasumpahan_1019 =
        new Node_2511531019("Pulau Pasumpahan",150,450);
Node_2511531019 museum_1019 =
        new Node_2511531019("Museum Adityawarman",250,400);

Node_2511531019[] nodes_1019 = {
        airManis_1019, jamGadang_1019, harau_1019, maninjau_1019,
        pagaruyung_1019, pantaiPadang_1019, bukittinggi_1019,
        sianok_1019, pasumpahan_1019, museum_1019
};
for(Node_2511531019 n_1019 : nodes_1019) {
    g_1019.addNode_2511531019(n_1019);
}

g_1019.addEdge_2511531019(airManis_1019, pantaiPadang_1019);
g_1019.addEdge_2511531019(airManis_1019, museum_1019);
g_1019.addEdge_2511531019(airManis_1019, pasumpahan_1019);

g_1019.addEdge_2511531019(pantaiPadang_1019, jamGadang_1019);
g_1019.addEdge_2511531019(pantaiPadang_1019, bukittinggi_1019);

g_1019.addEdge_2511531019(jamGadang_1019, bukittinggi_1019);
g_1019.addEdge_2511531019(jamGadang_1019, harau_1019);

g_1019.addEdge_2511531019(bukittinggi_1019, maninjau_1019);
g_1019.addEdge_2511531019(bukittinggi_1019, sianok_1019);

g_1019.addEdge_2511531019(manjau_1019, harau_1019);
g_1019.addEdge_2511531019(manjau_1019, pagaruyung_1019);

g_1019.addEdge_2511531019(harau_1019, pagaruyung_1019);

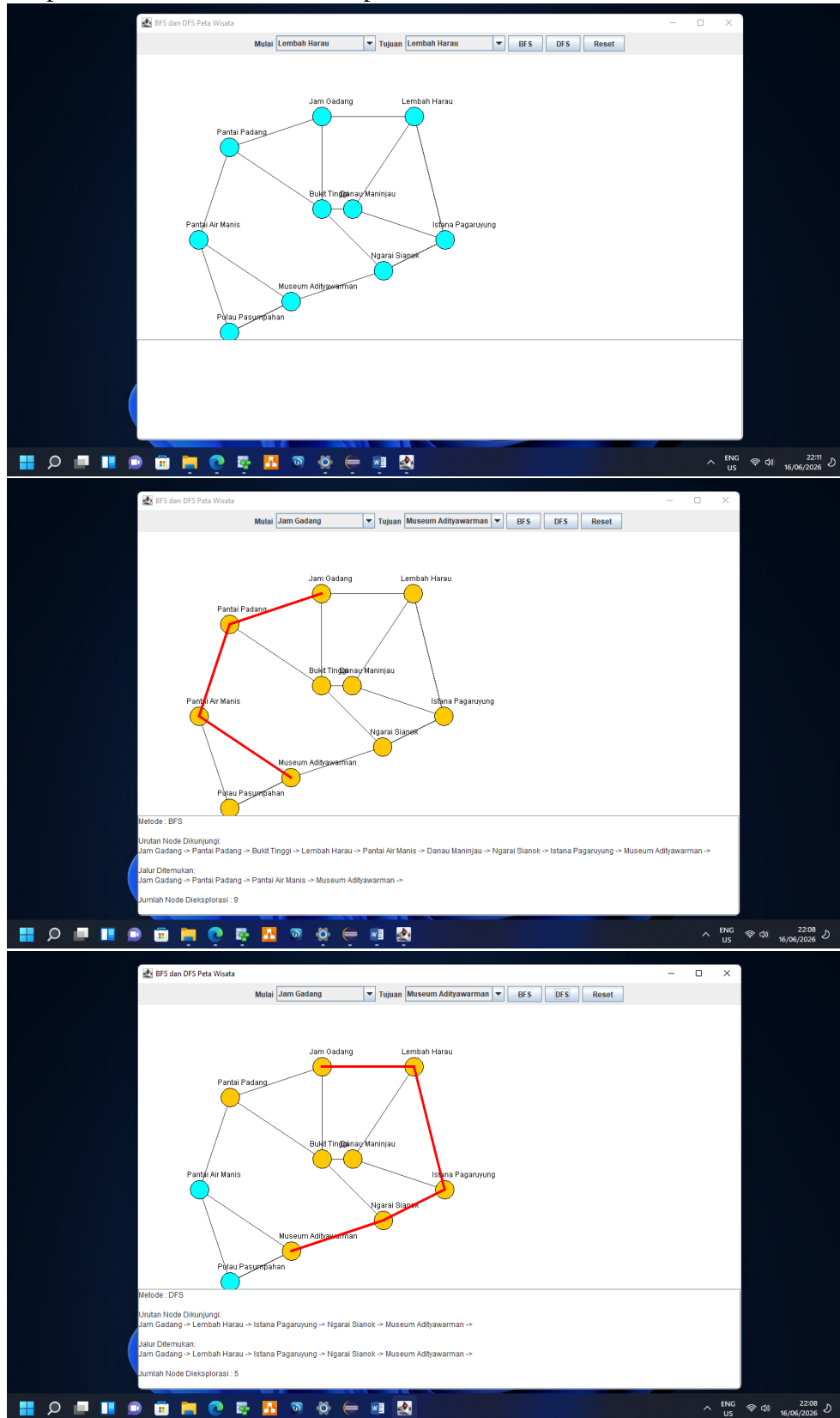
g_1019.addEdge_2511531019(sianok_1019, pagaruyung_1019);
g_1019.addEdge_2511531019(sianok_1019, museum_1019);

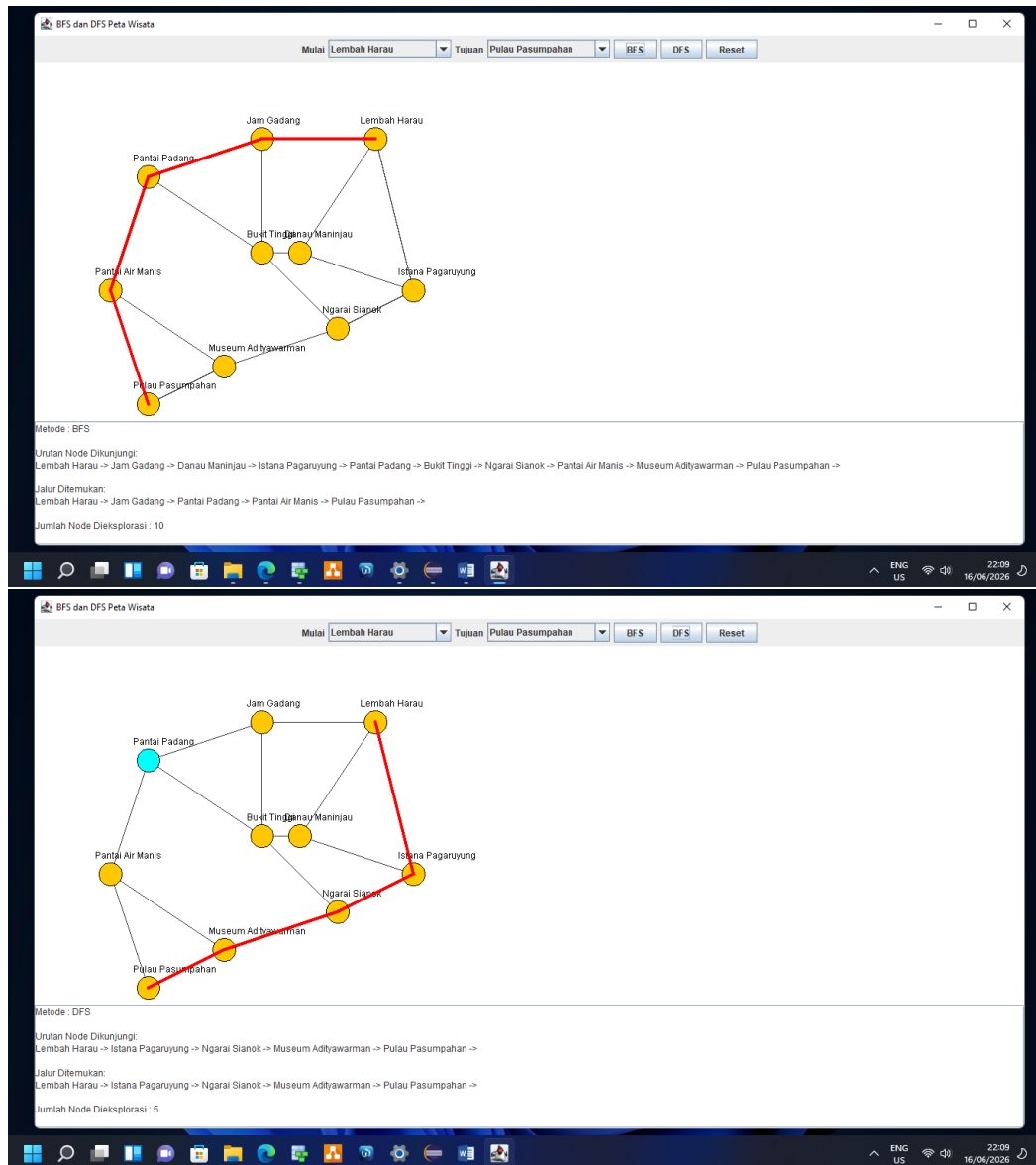
g_1019.addEdge_2511531019(museum_1019, pasumpahan_1019);
return g_1019;
}

public static void main(String[] args) {
    javax.swing.SwingUtilities.invokeLater(() -> {
        new PetaTempatWisata_2511531019();
    });
}
}

```

Output Kode Java GUI PetaTempatWisata:





2.4 Pembahasan

Praktikum ini menunjukkan bahwa Bahasa Java memudahkan proses membuat struktur data yang terdiri dari selector, mutator, dan lainnya. Dengan demikian, praktikum ini berhasil memberikan gambaran nyata tentang pentingnya Tipe Data Abstrak untuk struktur data dan driver sebagai eksekutor Tipe Data Abstrak untuk mendukung sistem informasi.

Bab 3 Kesimpulan

Praktikum ini berhasil menghasilkan struktur dalam data dengan menggunakan bahasa Java. Di dalam data memiliki struktur yang dapat menjalankan sebuah kasus kecil maupun besar untuk mendukung system informasi.